

COMPUTER ORGANIZATION
MODULE-II (Instruction & Control Unit)

Presented by:
Dr. Shasanka Sekhar Rout
Assistant Professor
EC Department

Module-II	**Multiprogramming and Multiprocessing**	R1	2	Introduction to pipeline operation, space-time diagram, speed-up ratio
	Control Unit	R1	1	Instruction types and formats
		https://www.bau.edu.jo/UserPortal/UserProfile/PostsAttach/43038_4306	3	Instruction cycle, execution of a complete instruction.
		R2	1	Hardwired and micro programmed control unit, unconditional and conditional branching.
		R1	1	Microinstruction with next address field, pre-fetching microinstructions, Concept of horizontal and vertical microprogramming.
	Memory	https://www.youtube.com/watch?v=YHfulm7Tub	2	Basic concept of Memory and its hierarchy,
		R1	2	RAM memories, 2D, 2 & 1/2D memory organization. ROM memories.
		R1	2	Cache memories: concept and design issues, performance, address mapping
		R1	1	page replacement algorithms
		R2	2	Virtual memory: concept and implementation.
	Input/Output	R2	1	Peripheral devices, I/O interface, I/O ports,
		R2	2	Interrupts: interrupt hardware, types of interrupts and exceptions.
		R2	1	Buses, bus architecture, types of buses and bus arbitration.
		R2	2	Modes of Data Transfer: Programmed I/O, interrupt initiated I/O and Direct Memory Access., I/O channels and processors.
		R2	1	Standard communication interfaces.

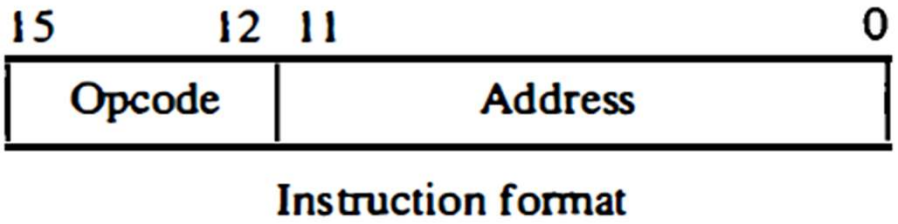
Instruction Code

- ✓ A computer instruction is a binary code that specifies a sequence of microoperations for the computer.
- ✓ Instruction codes together with data are stored in memory.
- ✓ An instruction code is a group of bits that instruct the computer to perform a specific operation.
- ✓ The most basic part of an instruction code is its operation part.
- ✓ The operation code must consist of at least n bits for a given 2^n (or less) distinct operations.
- ✓ An instruction code must therefore specify not only the operation but also the registers or the memory words where the operands are to be found, as well as the register or memory word where the result is to be stored.

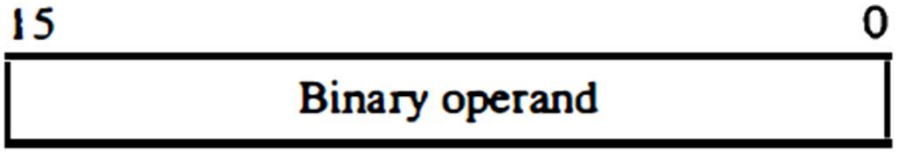
Stored Program Organization:

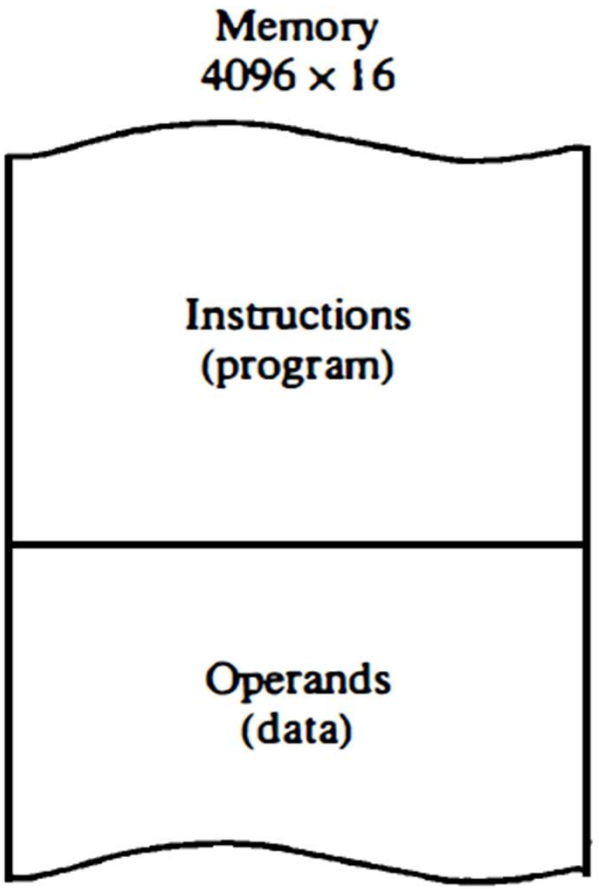
- ✓ The simplest way to organize a computer is to have one processor register and an instruction code format with two parts.
- ✓ Instructions are stored in one section of memory and data in another.
- ✓ For a memory unit with 4096 words, we need 12 bits to specify an address since $2^{12} = 4096$. If we store each instruction code in one 16-bit memory word, we have available four bits for the operation code (abbreviated opcode) to specify one out of 16 possible operations, and 12 bits to specify the address of an operand.
- ✓ The control reads a 16-bit instruction from the program portion of memory. It uses the 12-bit address part of the instruction to read a 16-bit operand from the data portion of memory. It then executes the operation specified by the operation code.

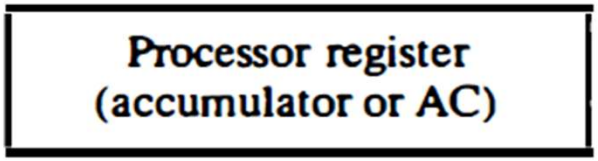
Stored Program Organization:



Instruction format

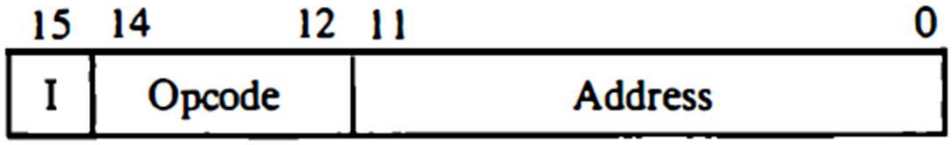






Indirect Address:

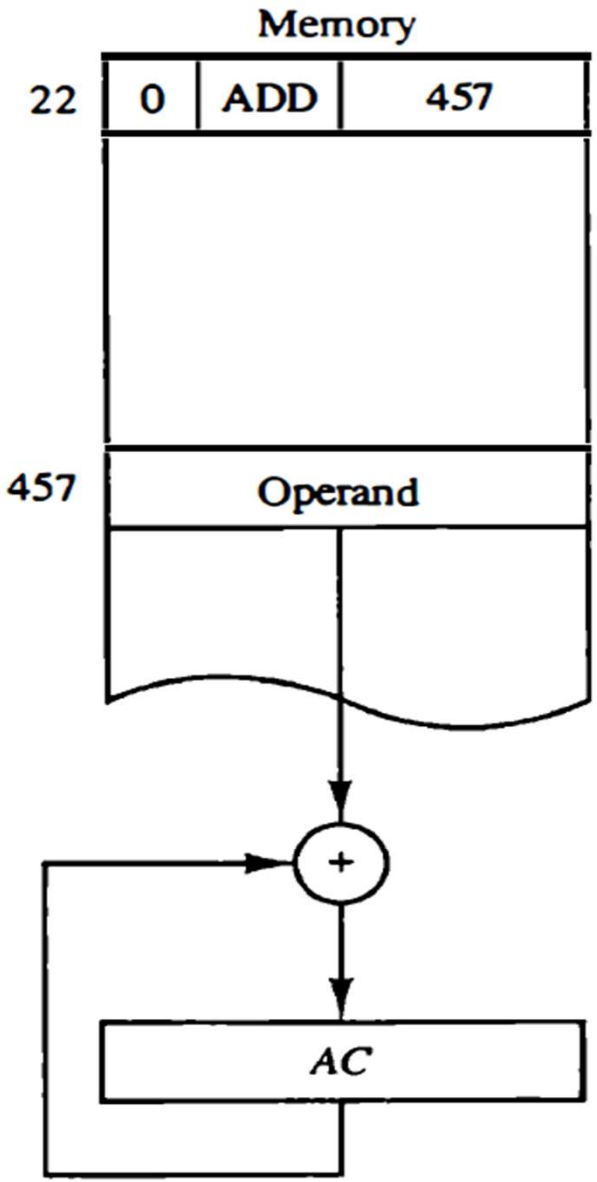
- ✓ In indirect address, the bits in the second part of the instruction designate an address of a memory word in which the address of the operand is found. One bit of the instruction code can be used to distinguish between a direct and an indirect address.
- ✓ Below instruction code format consists of a 3-bit operation code, a 12-bit address, and an indirect address mode bit designated by I. The mode bit is 0 for a direct address and 1 for an indirect address.



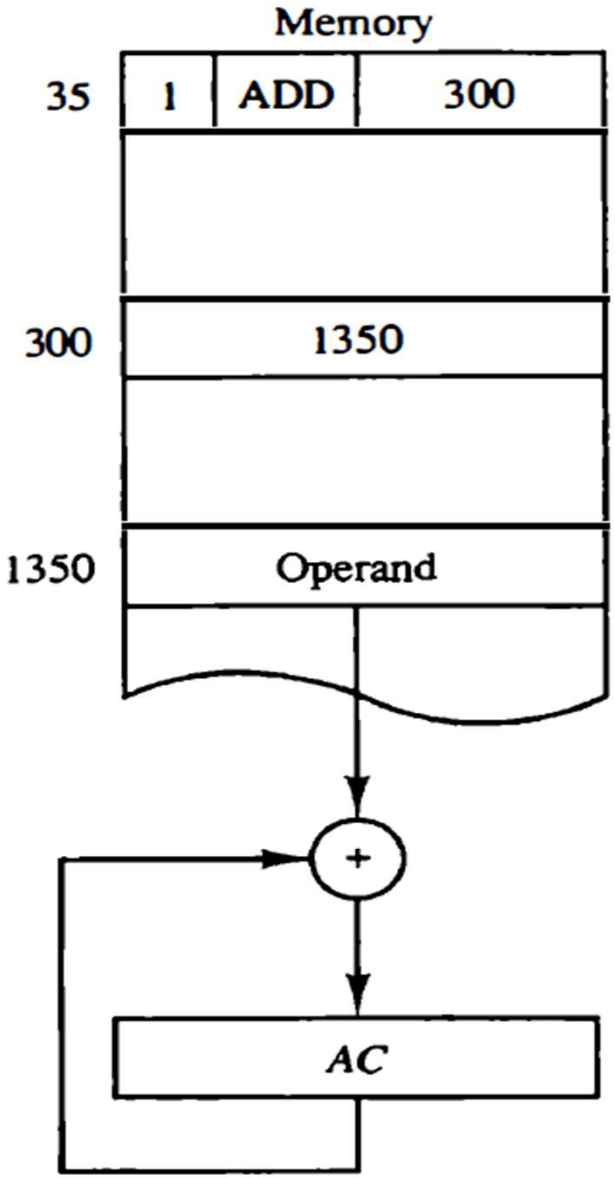
(a) Instruction format

- A direct address instruction as shown in next figure is placed in address 22 in memory. The I bit is 0, so the instruction is recognized as a direct address instruction. The opcode specifies an ADD instruction, and the address part is the binary equivalent of 457. The control finds the operand in memory at address 457 and adds it to the content of AC.

Indirect Address :



(b) Direct address



(c) Indirect address

Indirect Address:

- The instruction in address 35 has a mode bit I = 1. Therefore, it is recognized as an indirect address instruction. The address part is the binary equivalent of 300.
- The control goes to address 300 to find the address of the operand. The address of the operand in this case is 1350. The operand found in address 1350 is then added to the content of AC .
- The indirect address instruction needs two references to memory to fetch an operand. The first reference is needed to read the address of the operand; the second is for the operand itself. We define the effective address to be the address of the operand in a computation-type instruction or the target address in a branch-type instruction. Thus the effective address in direct and indirect address instruction is 457 and 1350 respectively.
- The memory word that holds the address of the operand in an indirect address instruction is used as a pointer to an array of data. The pointer could be placed in a processor register instead of memory as done in commercial computers.

Computer Registers:

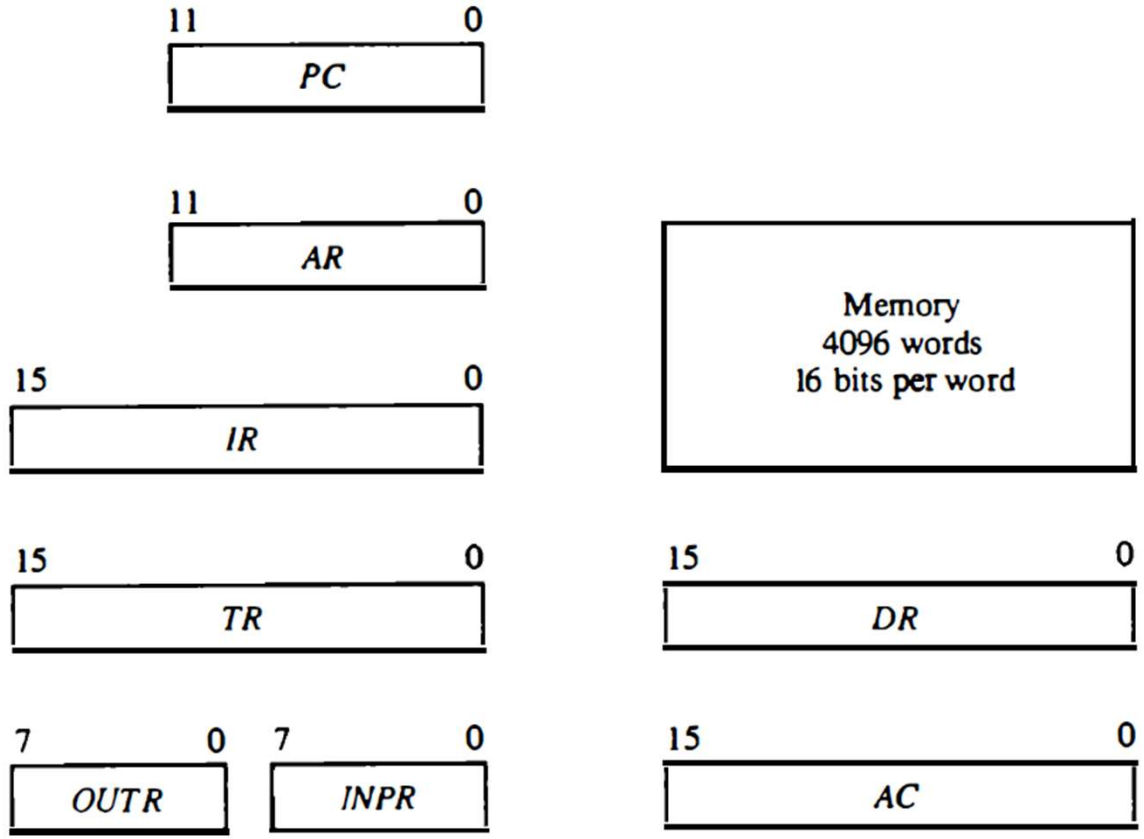
- It is necessary to provide a register in the control unit for storing the instruction code after it is read from memory. The computer needs processor registers for manipulating data and a register for holding a memory address.

Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

- The memory unit has a capacity of 4096 words and each word contains 16 bits. Twelve bits of an instruction word are needed to specify the address of an operand. This leaves three bits for the operation part of the instruction and a bit to specify a direct or indirect address.

Computer Registers:

- The data register (DR) holds the operand read from memory. The accumulator (AC) register is a general purpose processing register. The instruction read from memory is placed in the instruction register (IR). The temporary register (TR) is used for holding temporary data during the processing.

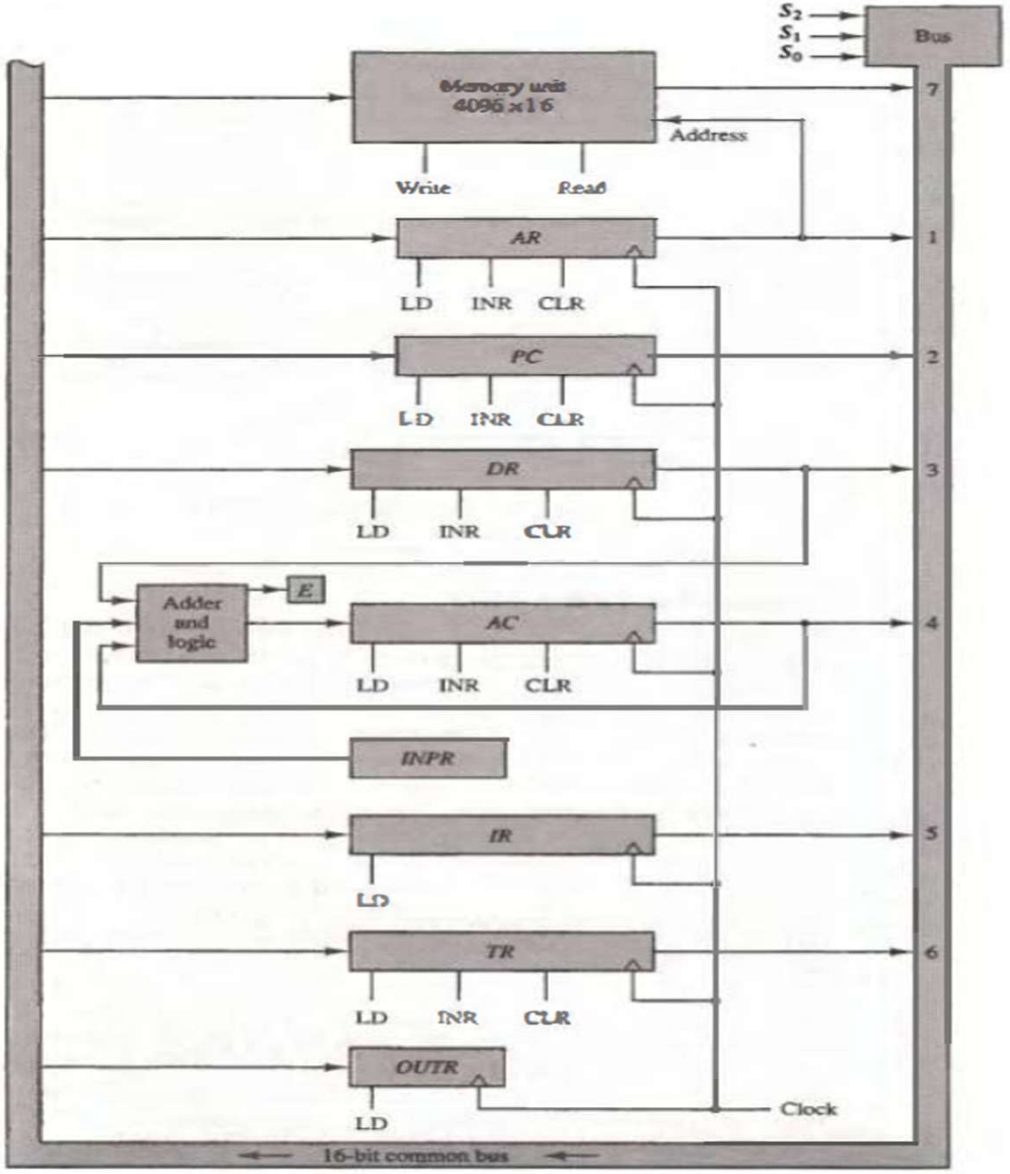


Computer Registers:

- The memory address register (AR) has 12 bits since this is the width of a memory address. The program counter (PC) also has 12 bits and it holds the address of the next instruction to be read from memory after the current instruction is executed. The PC goes through a counting sequence and causes the computer to read sequential instructions previously stored in memory.
- The address part of a branch instruction is transferred to PC to become the address of the next instruction. To read an instruction, the content of PC is taken as the address for memory and a memory read cycle is initiated. PC is then incremented by one, so it holds the address of the next instruction in sequence.
- Two registers are used for input and output. The input register (INPR) receives an 8-bit character from an input device. The output register (OUTR) holds an 8-bit character for an output device.

Common Bus System:

- A more efficient scheme for transferring information in a system with many registers is to use a common bus.



Common Bus System:

- The outputs of seven registers and memory are connected to the common bus. The specific output that is selected for the bus lines at any given time is determined from the binary value of the selection variables S_2, S_1, and S_0.
- The 16-bit outputs of DR are placed on the bus lines when $S_2S_1S_0 = 011$ since this is the binary value of decimal 3.
- The lines from the common bus are connected to the inputs of each register and the data inputs of the memory. The particular register whose LD (load) input is enabled receives the data from the bus during the next clock pulse transition. The memory receives the contents of the bus when its write input is activated. The memory places its 16-bit output onto the bus when the read input is activated and $S_2S_1S_0 = 111$.
- Four registers, DR, AC, IR, and TR, have 16 bits each. Two registers, AR and PC, have 12 bits each since they hold a memory address. When the contents of AR or PC are applied to the 16-bit common bus, the four MSB bits are set to 0's. When AR or PC receive information from the bus, only the 12 LSB bits are transferred into the register.

Common Bus System:

- The input register INPR and the output register OUTR have 8 bits each and communicate with the eight least significant bits in the bus. INPR is connected to provide information to the bus but OUTR can only receive information from the bus. This is because INPR receives a character from an input device which is then transferred to AC. OUTR receives a character from AC and delivers it to an output device. There is no transfer from OUTR to any of the other registers.
- The 16 lines of the common bus receive information from six registers and the memory unit. The bus lines are connected to the inputs of six registers and the memory. Five registers have three control inputs: LD (load), INR (increment), and CLR (clear).
- The input data and output data of the memory are connected to the common bus, but the memory address is connected to AR . Therefore, AR must always be used to specify a memory address. By using a single register for the address, we eliminate the need for an address bus that would have been needed otherwise. The content of any register can be specified for the memory data input during a write operation. Similarly, any register can receive the data from memory after a read operation except AC.

Common Bus System:

- The 16 inputs of AC come from an adder and logic circuit. This circuit has three sets of inputs. One set of 16-bit inputs come from the outputs of AC . They are used to implement register microoperations such as complement AC and shift AC.
- Another set of 16-bit inputs come from the data register DR. The inputs from DR and AC are used for arithmetic and logic microoperations, such as add DR to AC or AND DR to AC. The result of an addition is transferred to AC and the end carry-out of the addition is transferred to flip-flop E (extended AC bit). A third set of 8-bit inputs come from the input register INPR.
- Note that the content of any register can be applied onto the bus and an operation can be performed in the adder and logic circuit during the same clock cycle.
- The clock transition at the end of the cycle transfers the content of the bus into the designated destination register and the output of the adder and logic circuit into AC.

Common Bus System:

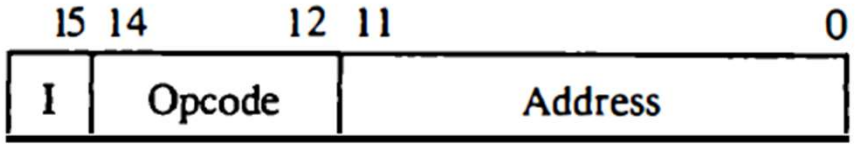
- For example, the two microoperations can be executed at the same time. This can be done by placing the content of AC on the bus (with $S_2S_1S_0 = 100$), enabling the LD (load) input of DR, transferring the content of DR through the adder and logic circuit into AC, and enabling the LD (load) input of AC, all during the same clock cycle.

- The two transfers occur upon the arrival of the clock pulse transition at the end of the clock cycle.

$$DR \leftarrow AC \text{ and } AC \leftarrow DR$$

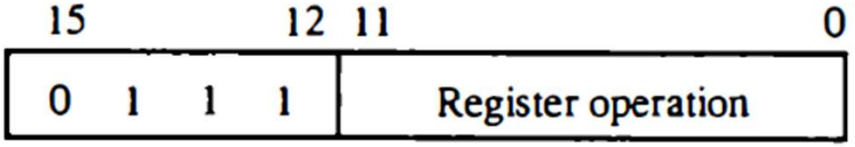
Computer Instructions:

- The basic computer has three instruction code formats, where each format has 16 bits. The operation code (opcode) part of the instruction contains three bits and the meaning of the remaining 13 bits depends on the operation code encountered. A memory-reference instruction uses 12 bits to specify an address and one bit to specify the addressing mode I. I is equal to 0 for direct address and to 1 for indirect address.



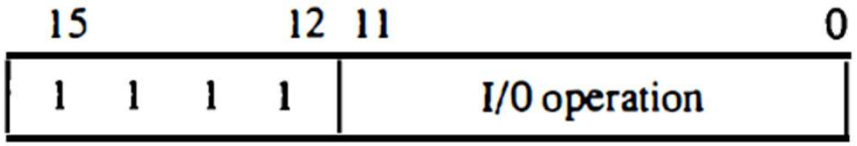
(Opcode = 000 through 110)

(a) Memory – reference instruction



(Opcode = 111, $I = 0$)

(b) Register – reference instruction



(Opcode = 111, $I = 1$)

(c) Input – output instruction

Computer Instructions:

- The register reference instructions are recognized by the operation code 111 with a 0 in the leftmost bit (bit 15) of the instruction. A register reference instruction specifies an operation on or a test of the AC register. An operand from memory is not needed; therefore, the other 12 bits are used to specify the operation or test to be executed.
- Similarly, an input-output instruction does not need a reference to memory and is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction. The remaining 12 bits are used to specify the type of input-output operation or test performed.
- The type of instruction is recognized by the computer control from the four bits in positions 12 through 15 of the instruction. If the three opcode bits in positions 12 though 14 are not equal to 111, the instruction is a memory-reference type and the bit in position 15 is taken as the addressing mode I.
- If the 3-bit opcode is equal to 111, control then inspects the bit in position 15. If this bit is 0, the instruction is a register-reference type. If the bit is 1, the instruction is an input-output type.

Computer Instructions:

- Only three bits of the instruction are used for the operation code. It may seem that the computer is restricted to a maximum of eight distinct operations.
- However, since register-reference and input-output instructions use the remaining 12 bits as part of the operation code, the total number of instructions can exceed eight. In fact, the total number of instructions chosen for the basic computer is equal to 25.
- In the instructions for the computer, the symbol designation is a three-letter word and represents an abbreviation intended for programmers and users.
- The hexadecimal code is equal to the equivalent hexadecimal number of the binary code used for the instruction. By using the hexadecimal equivalent we reduced the 16 bits of an instruction code to four digits with each hexadecimal digit being equivalent to four bits.

Computer Instructions:

- A memory-reference instruction has an address part of 12 bits. The address part is denoted by three x's and stand for the three hexadecimal digits corresponding to the 12-bit address. The last bit of the instruction is designated by the symbol I.
- When I = 0, the last four bits of an instruction have a hexadecimal digit equivalent from 0 to 6 since the last bit is 0. When I = 1, the hexadecimal digit equivalent of the last four bits of the instruction ranges from 8 to E since the last bit is 1.
- Register-reference instructions use 16 bits to specify an operation. The leftmost four bits are always 0111, which is equivalent to hexadecimal 7. The other three hexadecimal digits give the binary equivalent of the remaining 12 bits
- The input-output instructions also use all 16 bits to specify an operation. The last four bits are always 1111, equivalent to hexadecimal F.

Computer Instructions:

Symbol	Hexadecimal code $I = 0$	Hexadecimal code $I = 1$	Description
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC* in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear *AC*
CLE	7400		Clear *E*
CMA	7200		Complement *AC*
CME	7100		Complement *E*
CIR	7080		Circulate right *AC* and *E*
CIL	7040		Circulate left *AC* and *E*
INC	7020		Increment *AC*
SPA	7010		Skip next instruction if *AC* positive
SNA	7008		Skip next instruction if *AC* negative
SZA	7004		Skip next instruction if *AC* zero
SZE	7002		Skip next instruction if *E* is 0
HLT	7001		Halt computer
INP	F800		Input character to *AC*
OUT	F400		Output character from *AC*
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

Instruction Set Completeness:

- The set of instructions are said to be complete if the computer includes a sufficient number of instructions in each of the following categories:

1. Arithmetic, logical, and shift instructions
2. Instructions for moving information to and from memory and processor registers
3. Program control instructions together with instructions that check status conditions
4. Input and output instructions

Timing and Control:

- The timing for all registers in the basic computer is controlled by a master clock generator. The clock pulses are applied to all flip-flops and registers in the system, including the flip-flops and registers in the control unit.

- The clock pulses do not change the state of a register unless the register is enabled by a control signal.

- The control signals are generated in the control unit and provide control inputs for the multiplexers in the common bus, control inputs in processor registers, and microoperations for the accumulator.

- Control unit generates control signals using two organizations :
  - ✓ Hardwired Control Unit.
  - ✓ Micro-programmed Control Unit.

Hardwired Control Unit:

- Hardwired control units are implemented through use of sequential logic units or circuits like gates, flipflops, decoders in hardware.
- Hardwired control units are generally faster than micro-programmed designs.
- This architecture is preferred in reduced instruction set computers (RISC) as they use a simpler instruction set.

Advantages:

- Hardwired Control Unit is fast because control signals are generated by combinational circuits.
- The delay in generation of control signals depends upon the number of gates.
- The performances is high as compared to micro-programmed control unit.

Disadvantages:

- The control signals required by the CPU will be more complex.
- Modifications in control signal are very difficult. That means it requires rearranging of wires in the hardware circuit.
- It is difficult to correct mistake in original design or adding new features in existing design of control unit.

Micro-programmed Control Unit:

Advantages:

- The design of micro-program control unit is less complex because micro-programs are implemented using software routines.
- The micro-programmed control unit is more flexible because design modifications, correction and enhancement is easily possible.
- The new or modified instruction set of CPU can be easily implemented by simply rewriting or modifying the contents of control memory.
- The fault can be easily diagnosed in the micro-program control unit using diagnostics tools by maintaining the contents of flags, registers and counters.

Disadvantages:

- The micro-program control unit is slower than hardwired control unit. That means to execute an instruction in micro-program control unit requires more time.
- The micro-program control unit is expensive than hardwired control unit in case of limited hardware resources.
- The design duration of micro-program control unit is more than hardwired control unit for smaller CPU.

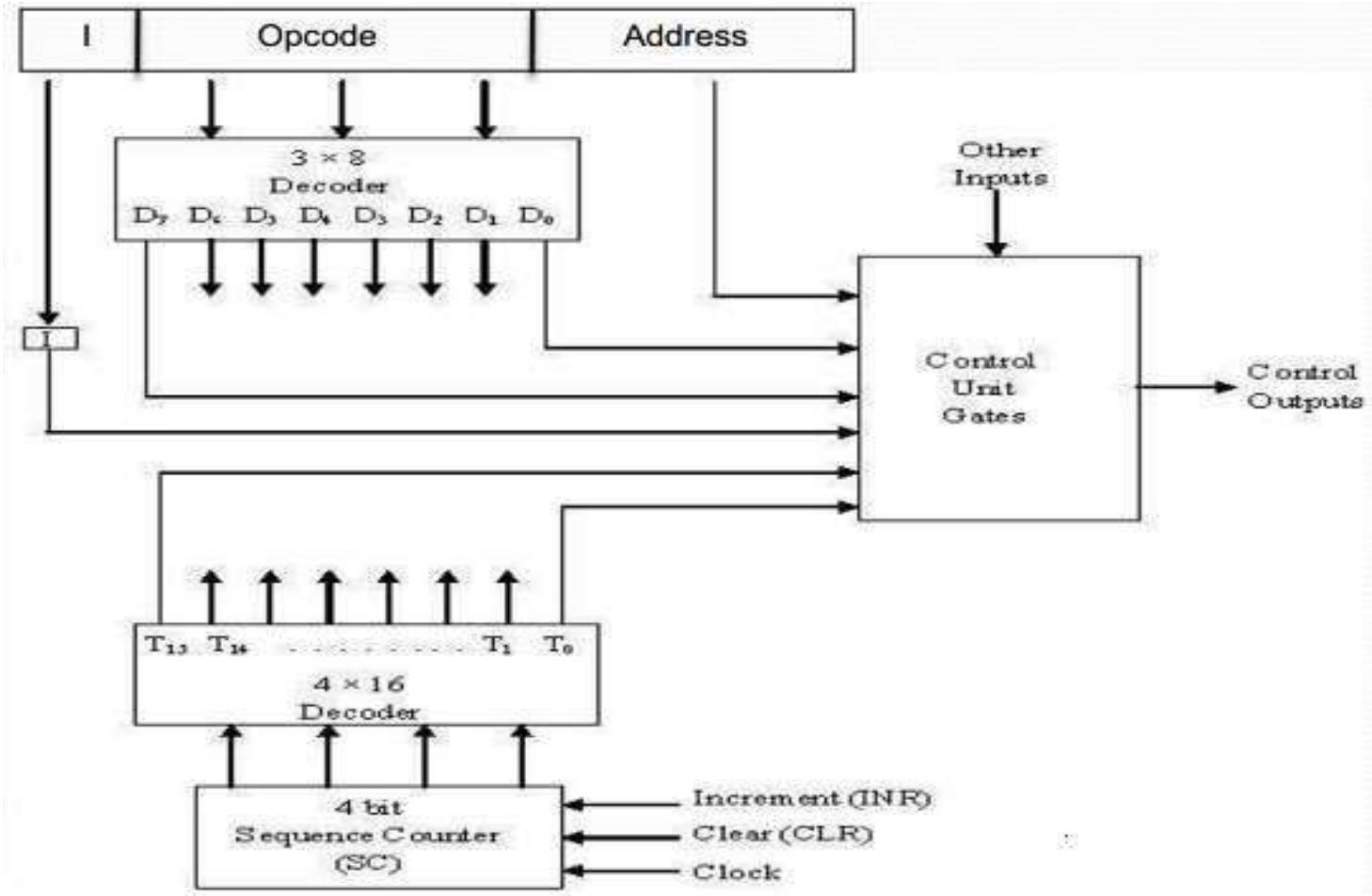
Comparison between Hardwired and Microprogrammed Control Unit:

ATTRIBUTE	HARDWIRED CONTROL UNIT	MICRO-PROGRAMMED CONTROL UNIT
SPEED	FAST	SLOW
COST OF IMPLEMENTATION	MORE	CHEAPER
FLEXIBILITY	NOT FLEXIBLE, DIFFICULT TO MODIFY FOR NEW INSTRUCTION	FLEXIBLE, NEW INSTRUCTIONS CAN BE ADDED
ABILITY TO HANDLE COMPLEX INSTRUCTION	DIFFICULT	EASIER
DECODING	COMPLEX	EASY
APPLICATION	RISC MICROPROCESSOR	CISC MICROPROCESSOR
INSTRUCTION SET SIZE	SMALL	LARGE
CONTROL MEMORY	ABSENT	PRESENT
CHIP AREA REQUIRED	LESS	MORE

Control Unit Architecture:

A control unit consist of:

- ❖ Instruction Register
- ❖ Number of Control Logic Gates
- ❖ Two Decoders
- ❖ 4-bit Sequence Counter



Control Unit:

- An instruction read from memory is placed in the instruction register (IR).
- The instruction register is divided into three parts: the I bit, operation code, and address part.
- First 12-bits (0-11) to specify an address, next 3-bits specify the operation code (opcode) field of the instruction and last left most bit specify the addressing mode I.
- I = 0 for direct address
- I = 1 for indirect address

- First 12-bits (0-11) are applied to the control logic gates.
- The operation code bits (12 – 14) are decoded with a 3 x 8 decoder.
- The eight outputs (D_0 through D_7) from a decoder goes to the control logic gates to perform specific operation.
- Last bit 15 is transferred to a I flip-flop designated by symbol I.

Control Unit:

- The 4-bit sequence counter SC can count in binary from 0 through 15.
- The counter output is decoded into 16 timing pulses T_0 through T_{15}.
- The sequence counter can be incremented by INR.
- Input or clear by CLR input synchronously.

- **Example:**

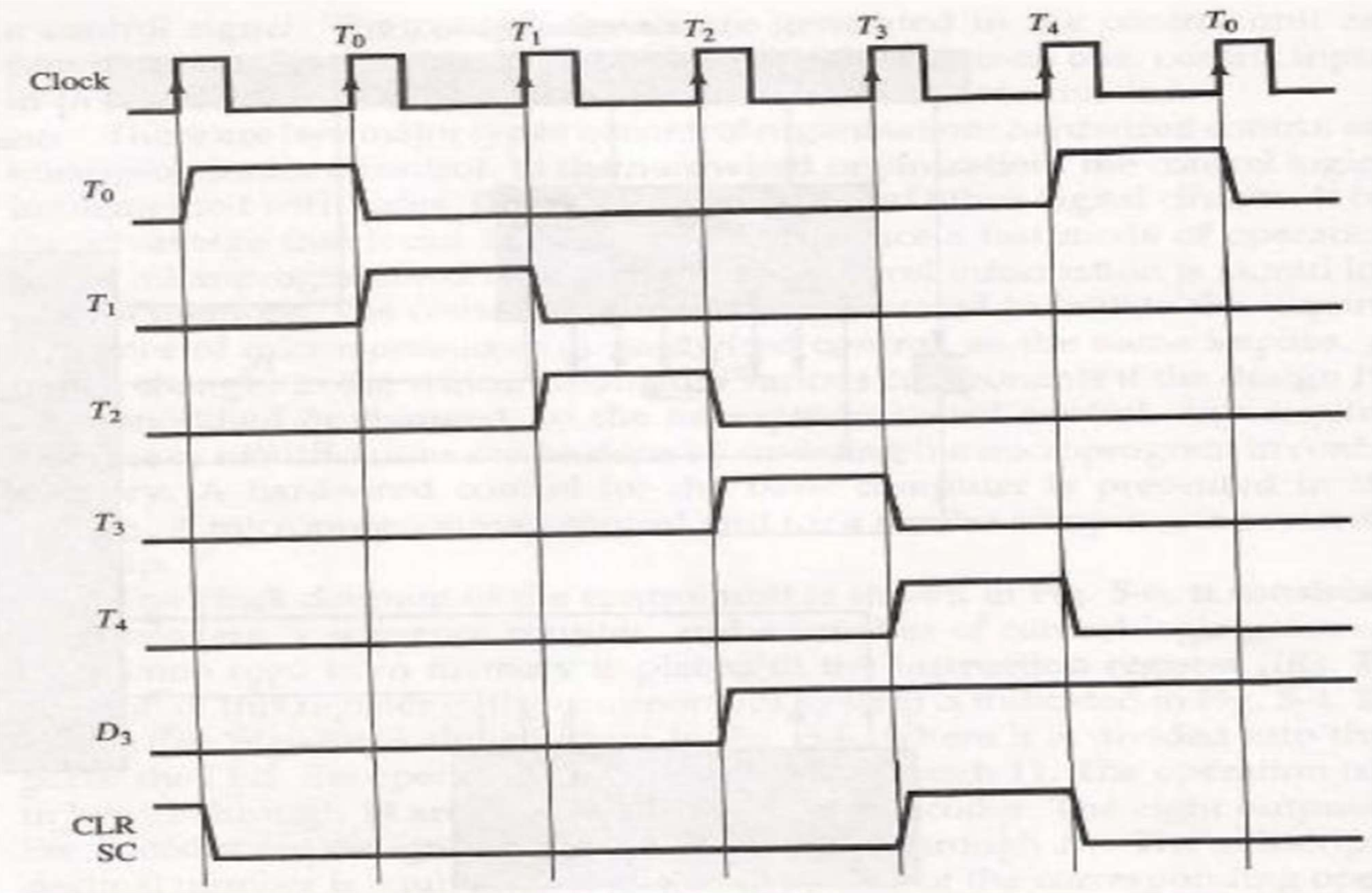
Consider the case where SC is incremented to provide timing signals T_0, T_1, T_2 , T_3, and T_4 in sequence. At time T_4 , SC is cleared to 0 if decoder output D_3 is active. This is expressed symbolically by the statement:

$$D_3\, T_4 : SC \leftarrow 0$$

- Output D_3 from the operation decoder becomes active at the end of timing signal T_2.
- SC is cleared when $D_3T_4 = 1$

Timing Diagram:

- The timing diagram shows the time relationship of the control signals.

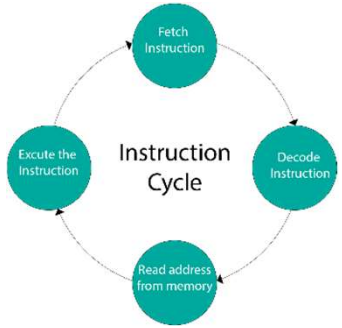


Instruction Cycle:

- A program residing in the memory unit of a computer consists of a sequence of instructions. These instructions are executed by the processor by going through a cycle for each instruction.

- In a basic computer, each instruction cycle consists of the following phases:

1. Fetch instruction from memory.
2. Decode the instruction.
3. Read the effective address from memory if the instruction has an indirect address.
4. Execute the instruction.



(This process continues indefinitely unless a HALT instruction is encountered)

- Instruction cycle is also defined as the combination of two cycles i.e. Fetch cycle (FC) and Execution cycle (EC).
- An ISA (Instruction set architecture) is a medium whereby a processor communicates with the human programmer.
- ISA acts as an interface between hardware & software and is divided into 2 parts:

RISC (Reduced Instruction Set Computer**)**

CISC (Complex Instruction Set Computer**)**

RISC	CISC
❖ It focuses on software	✓ It focuses on hardware
❖ Small number of instructions	✓ Large number of instructions
❖ Less addressing modes	✓ More addressing modes
❖ The program written for RISC architecture needs to take more space in memory(RAM)	✓ Needs less space in RAM
❖It has fixed instruction format	✓ Variable instruction format
❖ Low power consumption ❖ Uses more number registers ❖ Pipelining is easy ❖ It executes one instruction per clock cycle ❖ The execution time of RISC is very short. ❖ Ex: Alpha, ARC ARM, AVR, PIC	✓ High power consumption ✓ Less number of registers ✓ Pipelining is difficult ✓ It executes one instruction with more clock cycles ✓ The execution time of CISC is more. ✓ Ex: Motorola 6800 family, Intel x86 CPUs.

Fetch and Decode:

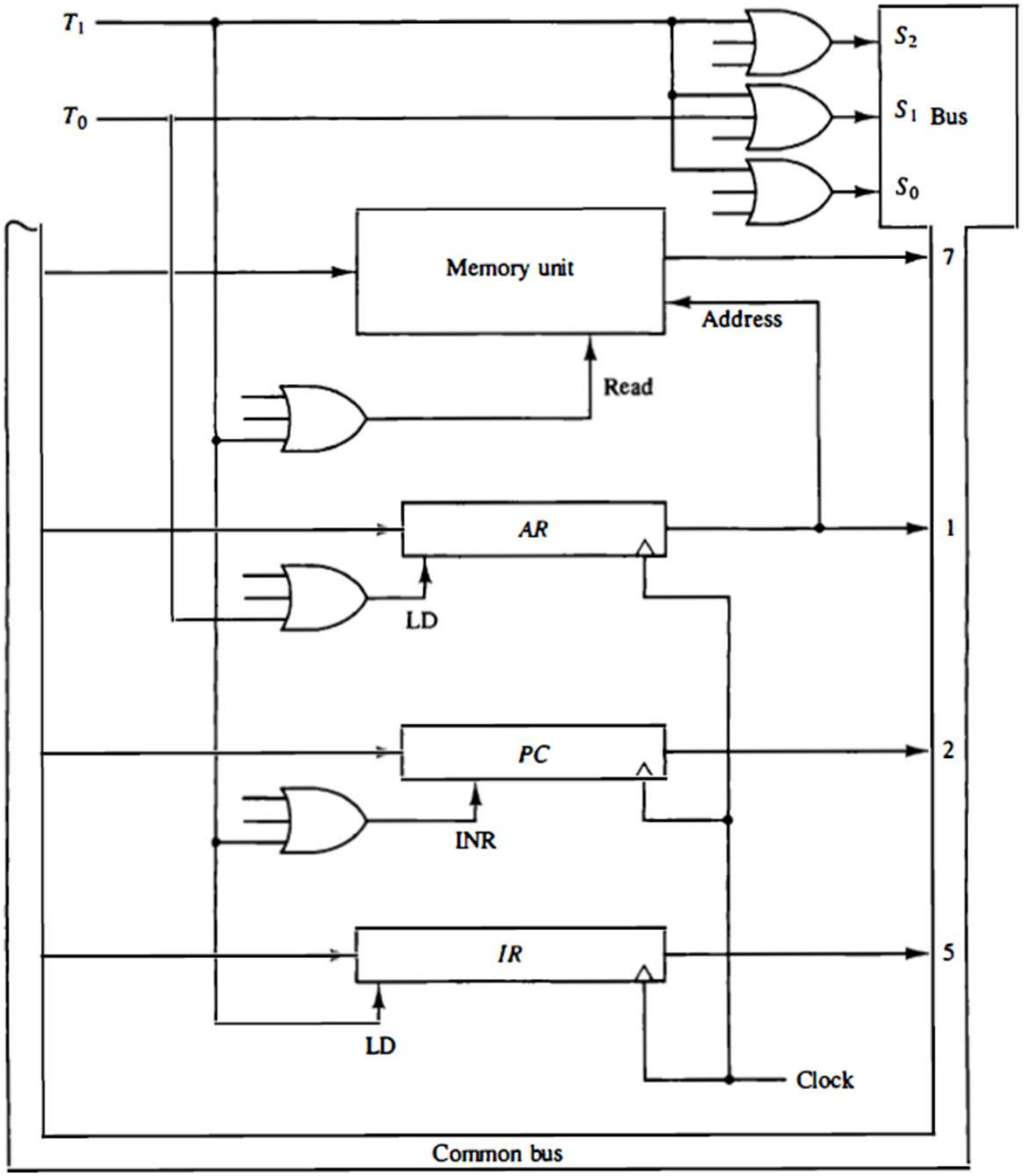
- Initially, the program counter PC is loaded with the address of the first instruction in the program. The sequence counter SC is cleared to 0, providing a decoded timing signal T_0. After each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T_0, T_1, T_2, and so on.
- The microoperations for the fetch and decode phases can be specified by the following register transfer statements.

T_0: AR ← PC
T_1: IR ← M[AR], PC ← PC + 1
T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)

- Since only AR is connected to the address inputs of memory, it is necessary to transfer the address from PC to AR during the clock transition associated with timing signal T_0. The instruction read from memory is then placed in the instruction register IR with the clock transition associated with timing signal T_1. At the same time, PC is incremented by one to prepare it for the address of the next instruction in the program. At time T_2, the operation code in IR is decoded, the indirect bit is transferred to flip-flop I, and the address part of the instruction is transferred to AR . Note that SC is incremented after each clock pulse to produce the sequence T_0, T_1, and T_2.

Fetch and Decode:

- Below Figure shows how the first two register transfer statements are implemented in the bus system.



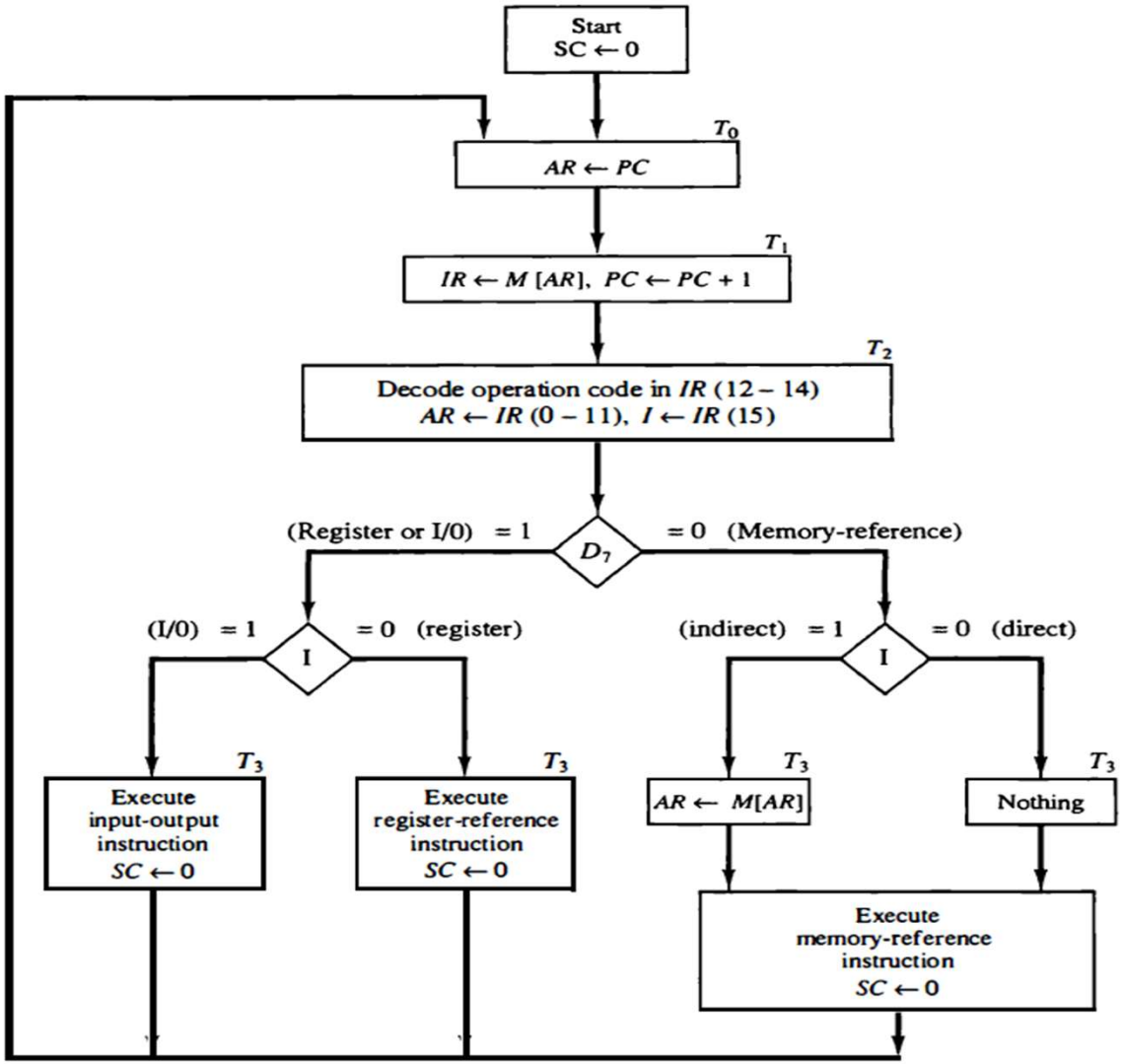
Fetch and Decode:

- To provide the data path for the transfer of PC to AR we must apply timing signal T_0 to achieve the following connection:
    1. Place the content of PC onto the bus by making the bus selection inputs $S_2S_1S_0$ equal to 010.
    2. Transfer the content of the bus to AR by enabling the LD input of AR.
- The next clock transition initiates the transfer from PC to AR since $T_0 = 1$. In order to implement the second statement

  T_1: IR ← M[AR], PC ← PC + 1
- It is necessary to use timing signal T_1 to provide the following connections in the bus system.
    1. Enable the read input of memory.
    2. Place the content of memory onto the bus by making $S_2S_1S_0 = 111$.
    3. Transfer the content of the bus to IR by enabling the LD input of IR.
    4. Increment PC by enabling the INR input of PC.
- The next clock transition initiates the read and increment operations since $T_1 = 1$.
- The figure duplicates a portion of the bus system and shows how T_0 and T_1 are connected to the control inputs of the registers, the memory, and the bus selection inputs. Multiple input OR gates are included in the diagram because there are other control functions that will initiate similar operations.

Determine the Type of Instruction:

- The timing signal that is active after the decoding is T_3. During time T_3, the control unit determines the type of instruction that was just read from memory. The below flowchart presents an initial configuration for the instruction cycle and shows how the control determines the instruction type after the decoding.



Determine the Type of Instruction:

- Decoder output $D_7 = 1$ if the operation code is equal to binary 111. From instruction code format we determine that if $D_7 = 1$, the instruction must be a register-reference or input-output type. If $D_7 = 0$, the operation code must be one of the other seven values 000 through 110, specifying a memory-reference instruction.
- Control then inspects the value of the first bit of the instruction, which is now available in flip-flop I. If $D_7 = 0$ and $I = 1$, we have a memory reference instruction with an indirect address. It is then necessary to read the effective address from memory. The microoperation for the indirect address condition can be symbolized by the register transfer statement.

$$AR \leftarrow M[AR]$$

- Initially, AR holds the address part of the instruction. This address is used during the memory read operation. The word at the address given by AR is read from memory and placed on the common bus. The LD input of AR is then enabled to receive the indirect address that resided in the 12 least significant bits of the memory word.

Determine the Type of Instruction:

- The three instruction types are subdivided into four separate paths. The selected operation is activated with the clock transition associated with timing signal T_3. This can be symbolized as follows:

$D'_7 IT_3$: $AR \leftarrow M[AR]$
$D'_7 I'T_3$: Nothing
$D_7 I'T_3$: Execute a register-reference instruction
$D_7 IT_3$: Execute an input–output instruction

- When a memory-reference instruction with I = 0 is encountered, it is not necessary to do anything since the effective address is already in AR. However, the sequence counter SC must be incremented when $D'_7T_3 = 1$, so that the execution of the memory-reference instruction can be continued with timing variable T_4. A register-reference or input-output instruction can be executed with the clock associated with timing signal T_3. After the instruction is executed, SC is cleared to 0 and control returns to the fetch phase with $T_0 = 1$.

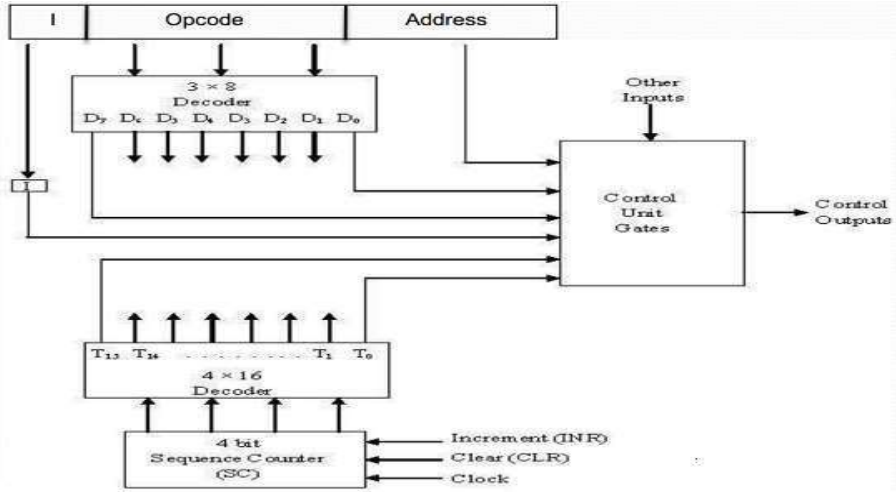
- Note that the sequence counter SC is either incremented or cleared to 0 with every positive clock transition. We will adopt the convention that if SC is incremented, we will not write the statement SC ← SC+1, but it will be implied that the control goes to the next timing signal in sequence. When SC is to be cleared, we will include the statement SC ← 0.

Register-Reference Instructions:

- The control functions and microoperations for the register-reference instructions and memory-reference are listed in below Table.

$D_7I'T_3 = r$ (common to all register-reference instructions)
$IR(i) = B_i$ [bit in $IR(0–11)$ that specifies the operation]

	r:	$SC \leftarrow 0$	Clear SC
CLA	rB_{11}:	$AC \leftarrow 0$	Clear AC
CLE	rB_{10}:	$E \leftarrow 0$	Clear E
CMA	rB_9:	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8:	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7:	$AC \leftarrow$ shr AC, $AC(15) \leftarrow E$, $E \leftarrow AC(0)$	Circulate right
CIL	rB_6:	$AC \leftarrow$ shl AC, $AC(0) \leftarrow E$, $E \leftarrow AC(15)$	Circulate left
INC	rB_5:	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0:	$S \leftarrow 0$ (S is a start–stop flip-flop)	Halt computer



- For example, the instruction CLA has the hexadecimal code 7800, which gives the binary equivalent 0111 1000 0000 0000.
- The first bit is a zero and is equivalent to I'.
- The next three bits constitute the operation code and are recognized from decoder output D_7.
- Bit 11 in IR is 1 and is recognized from B_{11}. The control function that initiates the microoperation for this instruction is $D_7I'T_3\ B_{11} = rB_{11}$.
- The execution of a register-reference instruction is completed at time T_3.
- The sequence counter SC is cleared to 0 and the control goes back to fetch the next instruction with timing signal T_0.

Memory-Reference Instructions:

- The decoded output D_i for i = 0, 1, 2, 3, 4, 5, and 6 from the operation decoder that belongs to each instruction is included in the below table.
- The effective address of the instruction is in the address register AR and was placed there during timing signal T_2 when I = 0, or during timing signal T_3 when I = 1.
- The execution of the memory-reference instructions starts with timing signal T4.
- The symbolic description of each instruction is specified in the table in terms of register transfer notation.

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], \quad E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

Microoperations that execute this instruction are:

- **AND to AC:**

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_0T_4: DR ← M [AR]
  D_0T_5: AC ← AC /\ DR, SC ← 0

- **ADD to AC:**

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_1T_4: DR ← M [AR]
  D_1T_5: AC ← AC + DR, E ← C_{out}, SC ← 0

- **LDA: Load to AC:**

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_2T_4: DR ← M [AR]
  D_2T_5: AC ← DR, SC ← 0

Microoperations that execute this instruction are:

- **STA: Store AC:**

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_3T_4: M [AR] ← AC, SC ← 0

- **BUN: Branch Unconditionally:**

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_4T_4: PC ← AR, SC ← 0

- **BSA: Branch and Save Return Address:**

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_5T_4: M[AR] ← PC, AR ←AR + 1
  D_5T_5: PC ←AR, SC ← 0

ISZ: Increment and Skip if Zero:

  T_0: AR ← PC
  T_1: IR ← M[AR], PC ← PC + 1
  T_2: D_0, ..., D_7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(l5)
  D_6T_4: DR ←M [AR]
  D_6T_5: DR ←DR + 1
  D_6T_6: M [AR] ←DR, if (DR = 0) then (PC ←PC + 1), SC ←0

Difference between Horizontal and Vertical micro-programmed Control Unit:

S. No	Horizontal micro-programmed CU	Vertical micro-programmed CU
1.	It supports longer control word.	It supports shorter control word.
2.	It allows a higher degree of parallelism. If degree is n, then n Control Signals are enabled at a time.	It allows a low degree of parallelism i.e., the degree of parallelism is either 0 or 1.
3.	No additional hardware is required.	Additional hardware in the form of decoders is required to generate control signals.
4.	It is faster than a Vertical micro-programmed control unit.	It is slower than a Horizontal micro-programmed control unit.
5.	It is more flexible than a vertical micro-programmed control unit.	It is less flexible than horizontal but more flexible than that of a hardwired control unit.
6.	A horizontal micro-programmed control unit uses horizontal micro-instruction, where every bit in the control field attaches to a control line.	A vertical micro-programmed control unit uses vertical micro-instruction, where a code is used for each action to be performed and the decoder translates this code into individual control signals.
7.	The horizontal micro-programmed control unit makes less use of ROM encoding than the vertical micro-programmed control unit.	The vertical micro-programmed control unit makes more use of ROM encoding to reduce the length of the control word.

THANK YOU

